IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Large Language Models (LLMs) : Part 1

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How AI, ML and DL differ and where each is used
- What Machine Learning is and how models learn from data
- The main types of Machine Learning, such as supervised and unsupervised learning
- A brief introduction to Deep Learning and Neural Networks
- How Reinforcement Learning teaches agents through rewards and interaction

Topics Covered

- The Challenge with Language and Context
- Introduction to Large Language Models (LLMs)
- How Machines Tried to Understand Language Before LLMs
- Evolution of LLMs (Rules → ML → Deep Learning → Transformers → LLMs)
- Traditional NLP v. LLMs
- Significance of LLMs and Real-World Applications
- Neural Networks and the Need for Them

Topics Covered

- RNNs and LSTMs (Sequences)
- Word Embeddings and Attention Mechanism
- Transition to Transformers and Foundation Models
- Key Terminology in LLMs
- LLM Architecture and Training Methods (Pre-training, Fine-tuning, RLHF)
- Limitations, Risks and Future of LLMs

The Challenge with Language and Context

The Challenge with Language

Computer Great at Numbers

- 2+2 = 4 (always)
- Millions of calculations in seconds
- Reliable, precise, consistent

But Language Is Different…

- Words have multiple meanings (“bank”)
- Context changes meaning (“cool”)
- Nuance, tone, intent → difficult

The Gap We Face..

- We generate massive text daily (emails, reports, chats, code)
- Computers can search, but not understand
- Searching or filtering ≠ understanding

Icebreaker: Why Language Trips Up Machines

1. Context Confusion

- You text **“I’m dying!”**
- Human → laughing too hard
- Computer → emergency call?

2. Gossip Trouble

- **“She killed it in the meeting!”**
- Human → success
- Computer → crime?

3. Information Overload

- 300 emails in your inbox, Humans skim & filter
- Computers just search keywords, no real understanding

4. Translation Twist

- Phrase: **“Break a leg!”**
- Human → good luck
- Computer → actual injury?

Question for Reflection

How do we make machines truly understand and work with human language?

Introduction to Large Language Models (LLMs)

Introduction to Large Language Models

What They Are

Is a new class of AI systems built to work with human language

Trained on massive text data → books, websites, code, conversations

Equipped with billions of parameters to capture patterns, context and meaning

Introduction to Large Language Models

What They Can Do

Understand language → context, nuance, intent

Generate language → answers, summaries, translations, code

Bridge the gap: machines that think in words, not just numbers

Introduction to Large Language Models (LLMs)

In simple words

LLMs are like the “translator friend” who finally helps computers speak human.

Large Language Models

Technical View	Non-Technical View
A type of AI model based on deep learning (transformers)	A super-smart autocomplete for language
Trained on massive datasets (trillions of words)	Exposed to books, articles, websites and documents
Has billions+ parameters that capture patterns of language and reasoning.	Learns how words and ideas usually connect in human language.
Capable of performing text generation, summarisation, Q&A, code writing, translation, etc.	Capable of predicting the most meaningful next words to respond naturally

How Machines Tried to Understand Language Before LLMs

How Machines Tried to Understand Language Before

Rules → If/else patterns, good for simple tasks but brittle

Traditional NLP → Count words/statistics, but missed meaning & context

Early Deep Learning → Better at sequences, but struggled with long text

LLMs Today → Handle context, nuance and scale

How Machines Tried to Understand Language Before

Why This Matters

Solved earlier problems but introduced new limitations

Represented the culmination of decades of NLP progress

Required comparison with traditional approaches to appreciate LLMs

Evolution of LLMs

Evolution of LLM

Evolution: Rules → ML → Deep Learning → Transformers → LLMs

Rules-Based

Pre-2000s

- Hand-written rules: “Fragile, hard to scale
- If word = X, then Y”

Machine Learning

2000s

- Models (Naïve Bayes, SVMs) learned from data
- Still relied on manual features

Deep Learning

(2010s)

- Word embeddings (Word2Vec, GloVe) captured meaning
- RNNs/LSTMs improved sequences but weak at long context

Transformers

(2017 →)

- “Attention is All You Need” → full context, parallel training
- Scaled massively on GPUs/TPUs

LLMs

(2020s →)

- Trained on trillions of tokens, billions of parameters
- Handle reasoning, conversation, coding, multi-tasking
- Foundation models (GPT, PaLM, LLaMA) across industries

Traditional NLP v. LLMs

Traditional NLP v. LLMs

Aspect	Traditional NLP Models	Large Language Models (LLMs)
Training Data	Small, task-specific datasets (e.g., for sentiment or translation)	Massive text corpora from books, websites, code, etc.
Task Handling	One model per task (translation model ≠ summarisation model)	One model can handle multiple tasks via prompts
Flexibility	Limited; needs retraining for new tasks	Highly flexible; same model adapts to many tasks
Knowledge Scope	Narrow, domain-specific	Broad, covers many domains and topics
Performance	Good on trained tasks, poor generalisation	Strong generalisation, human-like fluency
Examples	Word2Vec, GloVe, RNNs, LSTMs	GPT, LLaMA, PaLM, Claude

Significance of LLMs and Real-World Applications

Significance of LLMs

Context Understanding

- Captures meaning beyond simple pattern matching
- Example: LLMs can distinguish “bank” as institution vs “bank” as river’s edge
- Delivers more relevant and accurate answers

Fluent Generation

- Is trained on billions of text examples to produce smooth, human-like language
- Provides natural, coherent, non-robotic responses

Wide Applications

- Is used across domains like chatbots, search engines, education, healthcare, coding assistants, creative writing and customer support
- Adapts to many tasks instead of being limited to one

Why Are They Called Large Language Models?

Large Parameters

- Billions → trillions of weights (vs. millions in small models)
- More capacity to capture complex language patterns

Large Training Data

- Trained on trillions of tokens (books, web, code, conversations)
- Covers many domains → broad generalisation

Large Compute

- Massive GPU/TPU clusters, weeks/months of training
- Scale enables emergent reasoning abilities

"Large" = large parameters + large data + large compute → powerful general-purpose language capabilities

Real-Life Examples of LLM

Use Case	Example Tool/Platform	Description
Writing Assistance	Grammarly	Checks grammar, spelling and improves writing style
Customer Support	Chatbots (e.g. Zendesk)	Answers FAQs and resolves support queries automatically
Language Translation	DeepL, Google Translate	Provides accurate instant language translation
Content Generation	Copy.ai, Jasper	Creates marketing text, social posts and blogs
Legal Document Review	Evisort, Lexion	Summarises, analyses and manages legal contracts
Healthcare	Med-PaLM	Assists with clinical questions and medical summaries
Education	Duolingo Max	Offers personalised language learning and feedback

Real-Life Analogy

Imagine Teaching a Child To Write and Understand Stories

Begin with simple words and short sentences

Progress to reading books and writing paragraphs

Advance to crafting full stories with depth and style

Neural Networks and the Need for Them

Need for Neural Networks

The Old Way: Teaching Every Rule

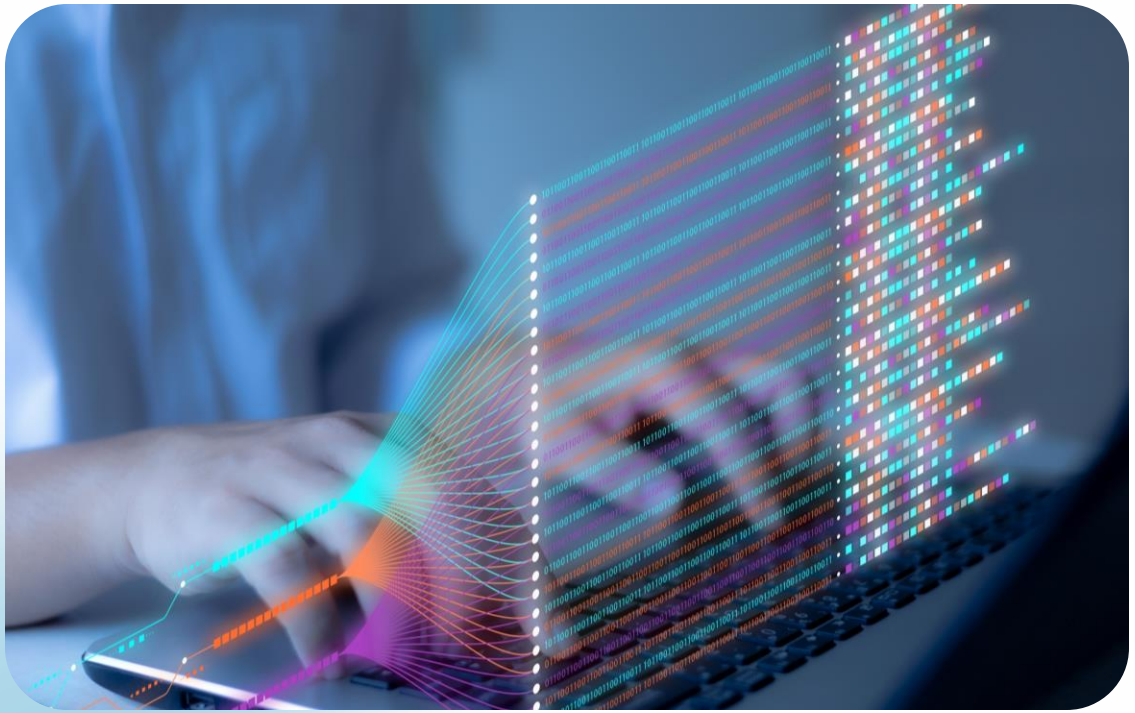
For training a child to recognise animals, you might say:

- “If it has feathers → it’s a bird.”
- “If it has four legs → it’s a dog.”

Works at times but fails with exceptions like penguins and bats

Need for Neural Networks

Challenges



- Real-world data (language, images, sounds) is too complex for rigid rules.
- Traditional ML still needed humans to tell the computer what to look at manually.
- The New Way: Learning From Examples
- Instead of coding rules, we show the system lots of examples.
- Like a child, it learns patterns automatically.
- That's the idea of Neural Networks → machines that learn features and rules by themselves.

Need for Neural Networks

The New Way: Learning From Examples

Show the system examples instead of coding rules

Learn patterns automatically, like a child

Idea of Neural Networks → machines that learn features and rules by themselves

Neural networks learn patterns from data, avoiding thousands of hand-written rules.

Neural Networks

The Core Idea

Is a way for a computer to learn patterns from data

Is inspired by how the human brain works with neurons

Data flows through layers of tiny “nodes” (neurons)

Each node makes a small decision → together they make a big decision

Neural Networks

Step-by-Step

Input layer → Raw data goes in (like words, images, numbers)

Output layer → Produces a prediction/answer (e.g., “spam” or “not spam”, “cat” or “dog”)

Step One | **Step Two** | **Step Three**

Hidden layers → Neurons process the data by combining weights (importance) and activation (trigger)

Neural Networks

Simple Analogy

A team solving a problem.

Each person scores (yes/no, strong/weak) part of the information.

The next team combines and refines the score.

The last person gives the final decision.

Neural Network Example: Email Spam Filter

Step 1 — Input Layer: An incoming email comes in

The input is broken into features, e.g.:

- Contains the word “free”?
- Has all caps subject?
- Too many exclamation marks?
- Sender domain?

Neural Network Example: Email Spam Filter

Step 2 — Hidden Layers: Each “neuron” checks a mix of features

Signals are weighted and neurons pass their scores forward as follows:

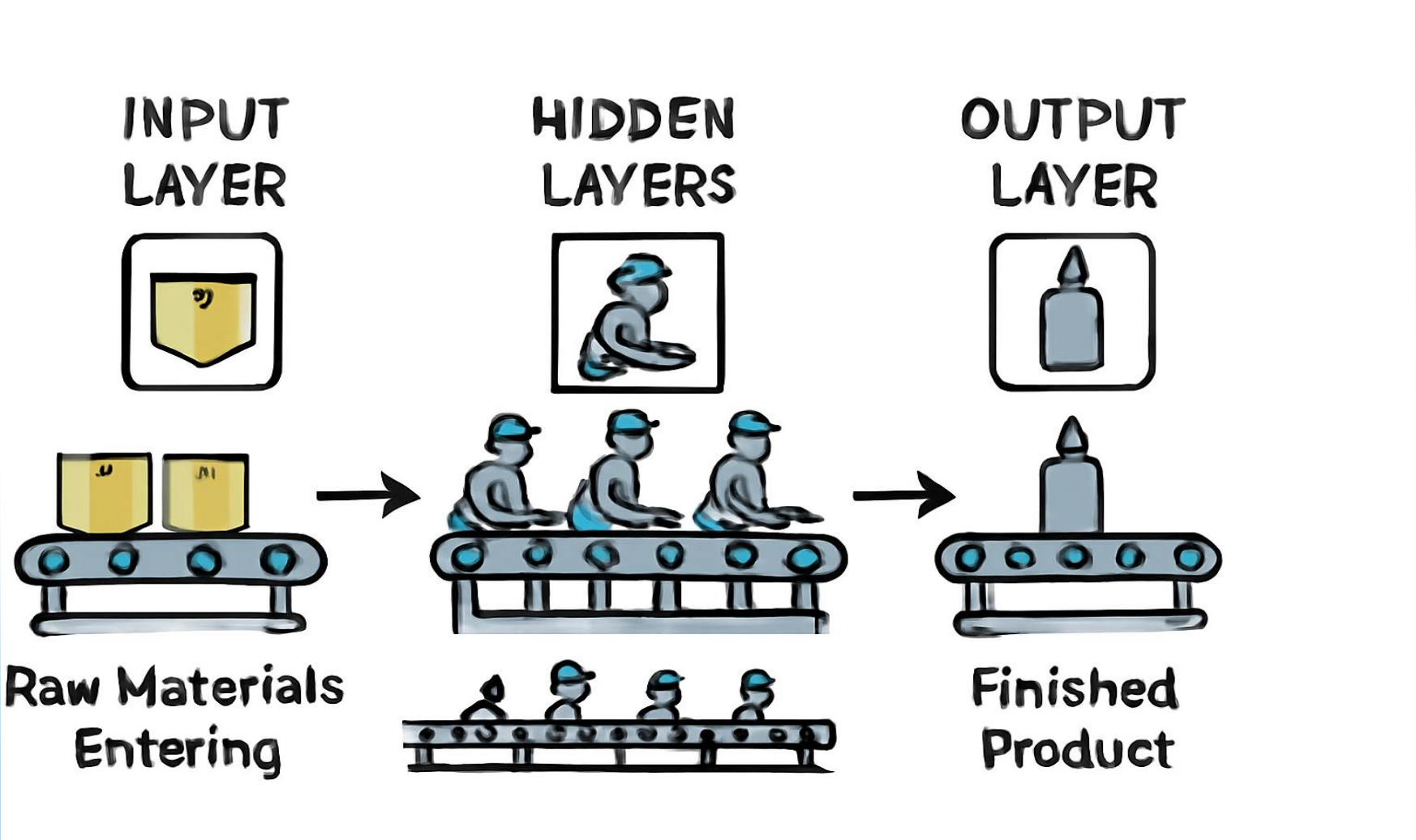
- Neuron A says → “Yes, looks suspicious because of ‘free’ + all caps.”
- Neuron B says → “Not sure, sender domain looks normal.”
- Neuron C says → “Multiple exclamation marks, probably spam.”

Neural Network Example: Email Spam Filter

Step 3 — Output Layer: All signals combine

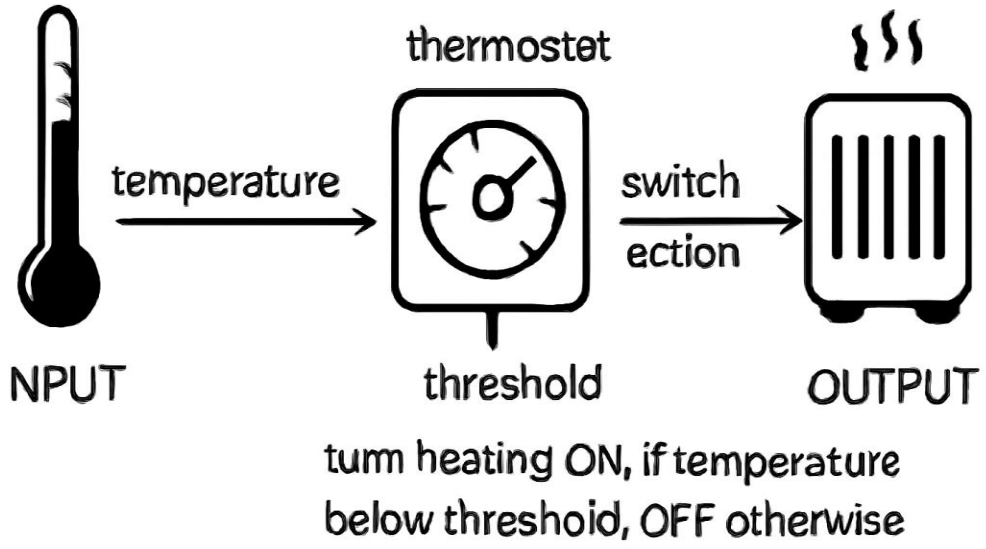
- Final decision: Spam = 0.9 probability (very likely spam).
- The system moves the email to your spam folder.

Neural Network



Neural Network

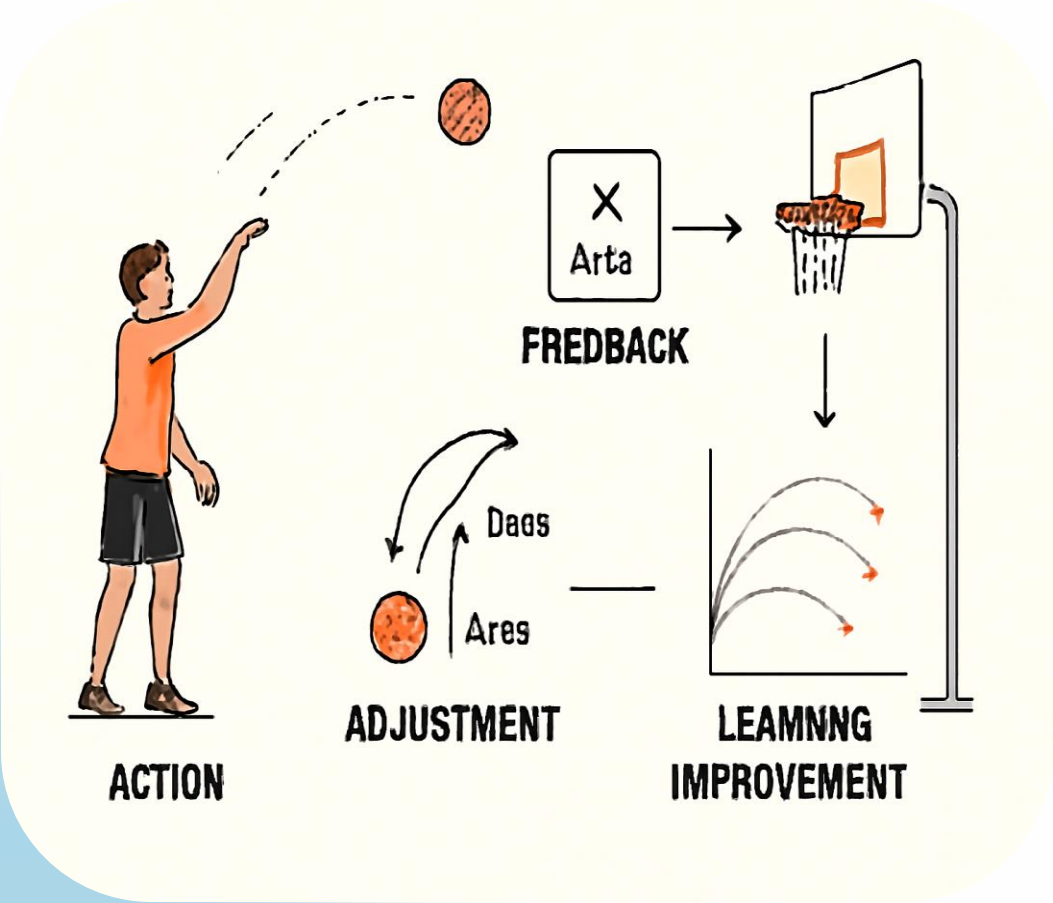
Activation Function



- Decides whether a neuron should “fire” (send information) based on input
- Adds non-linearity to help the network learn complex patterns
- Common types: ReLU, sigmoid, tanh

Neural Network

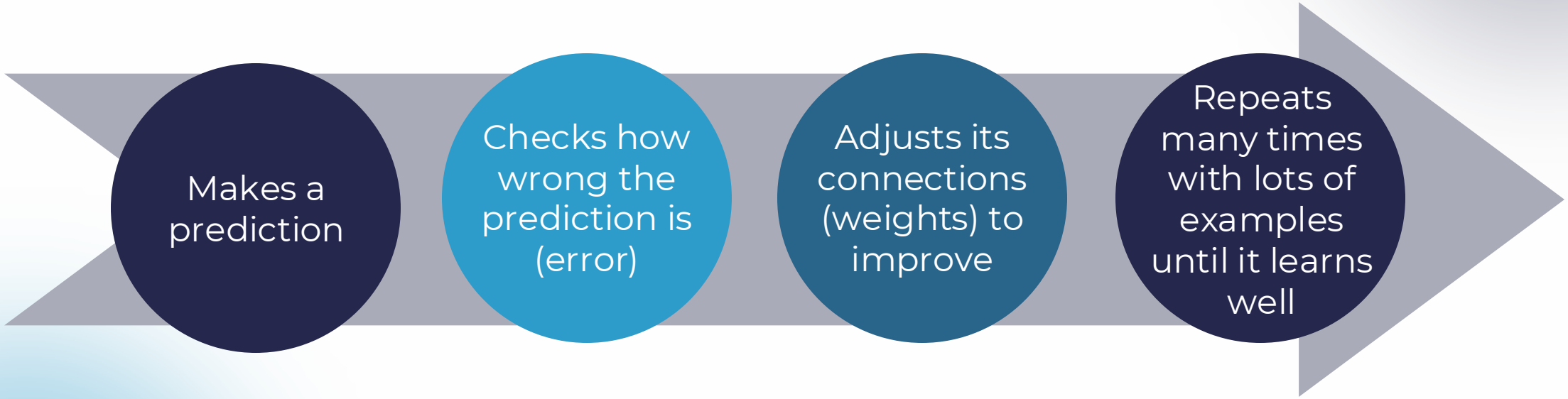
Training



- Adjust "neuron connections" to reduce errors
- Learns tasks like image recognition and language understanding

Neural Network

Training



Neural Network

Summary Table

Concept	Basic Meaning	Real-life Analogy
Layers	Steps in data processing	Factory assembly line
Activation Function	Decides if neuron should fire	Thermostat turning heating on/off
Training	Learning by correcting mistakes	Practicing basketball shots

Limitations of Basic Neural Networks

Weak with Sequences

- Treats inputs independently
- Struggles with order of words (e.g., “dog bites man” vs. “man bites dog”)

Limited Memory

- Struggles with long-term dependencies in text or speech
- Forgets earlier context as input grows

Feature Dependence

- Relies on heavy preprocessing and feature design

Training Challenges

- Requires large labelled datasets
- Is slow, computationally expensive as networks deepen

RNNs and LSTMs (Sequences)

RNNs & LSTMs (Sequences)

Concept

Regular neural networks	Recurrent Neural Networks (RNNs)	Long Short-Term Memory (LSTMs)
Treat inputs independently, failing to handle sequences	Pass information forward using memory	Remember information across longer sequences

Example
Predict the next word: *"I am going to the ___"*.
RNN "remembers" the earlier words *"going to the"*, to predict "store" correctly.

RNNs & LSTMs (Sequences)

Limitations

- Struggle with very long sentences (forget context far back)
- Slow to train because they process words step by step

Word Embeddings and Attention Mechanism

Word Embeddings (Word2Vec, GloVe)

Concept

Earlier models

- Treated words as unrelated symbols

Word embeddings

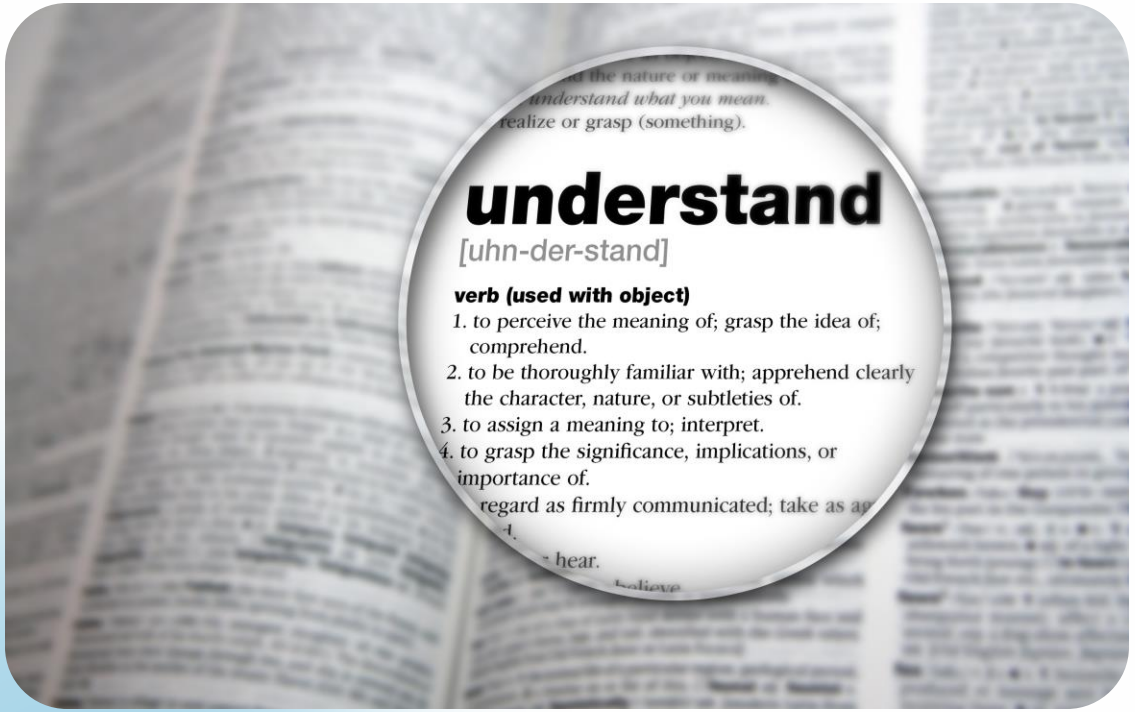
- Turned words into dense vectors capturing meaning
- Display similar words close together in vector space

Example

- *king – man + woman ≈ queen*
- Embeddings “understand” relationships beyond spelling

Word Embeddings (Word2Vec, GloVe)

Limitation



- Each word has only one fixed vector
- No sense of context — “bank” (river vs. finance) has the same embedding

Attention Mechanism

Concept

- **Translation and text tasks**: Some words matter more than others
- **Attention mechanism:** Focuses on key words instead of all equally
- **Spotlight analogy:** Highlights the most important parts of a sentence

Example
Translating *"The cat sat on the mat"* → the model pays more attention to *"cat"* when deciding how to translate the subject.

Attention Mechanism

Strength

- Handles long sentences better
- Helps models capture relationships between distant words

Limitation

- Originally just an add-on for RNNs/LSTMs → still limited until transformers arrived

Transition to Transformers and Foundation Models

Transformers | 2017 →

Concept

“Attention Is All You Need”

- Has replaced RNNs with a fully attention-based model

Transformers

- Looks at the entire sentence in parallel, not step-by-step.
- Makes training much faster and allows capturing long-range dependencies

Example:
Summarising a news article: the transformer can “see” the first and last sentence at the same time, maintaining full context.

Transformers | 2017 →

Strength

- Scales massively with GPUs/TPUs
- Is a base for today's modern language models

Large Language Models (LLMs) | 2020s →

Concept

Are transformers trained at massive scale

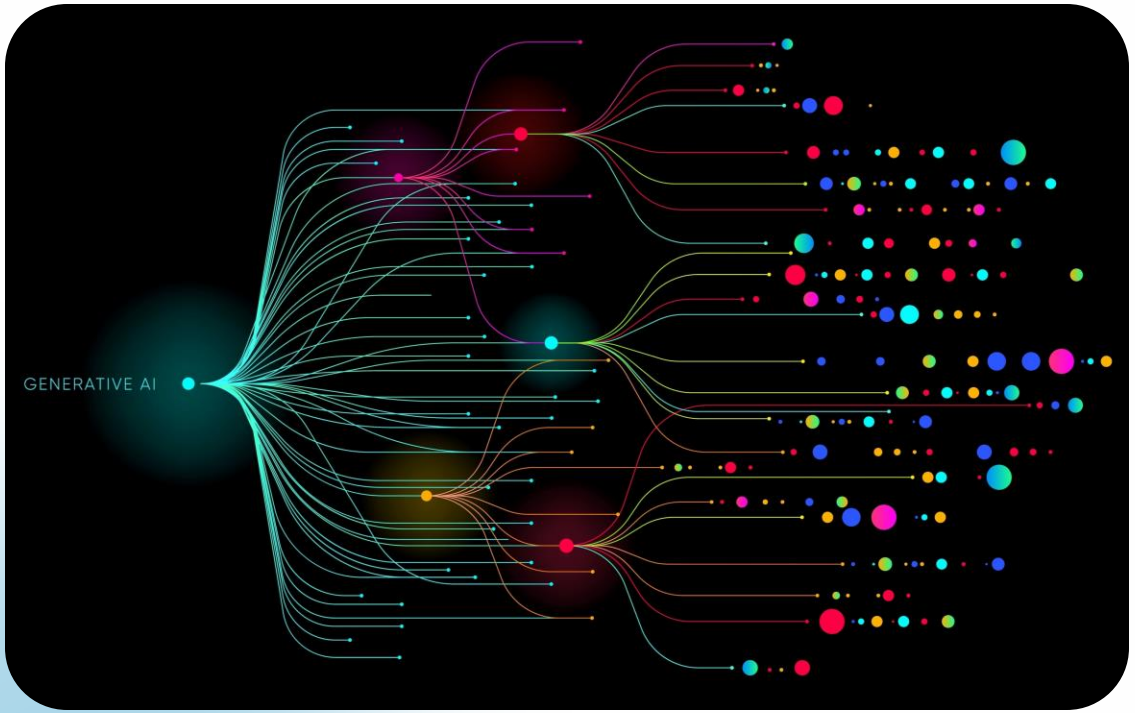
Trained on trillions of words with billions of parameters

Perform many tasks without being retrained
(reasoning, conversation, code)

Example:
ChatGPT answering a medical-style question, writing code or summarising 50 pages — all in one system.

Large Language Models (LLMs) | 2020s →

Strengths



- Exhibit emergent abilities: reasoning, multi-tasking, cross-industry adaptation
- Fine-tune foundation models (GPT, PaLM, LLaMA) for specific tasks

Key Terminology in LLMs

Key Terminology in LLMs

Token

The smallest unit of text the model processes (like pieces of words).

Example: “unbelievable” → “un”, “believe”, “able”.

Parameters

The “memory knobs” the model tunes during training.

Billions of parameters = model capacity.

Embedding

→ A way of representing words/phrases as numbers in a vector space, capturing meaning.

Example: “dog” is closer to “cat” than “car” in embedding space.

Context Window

The amount of text the model can “see” at once.

Example: GPT-3 saw ~2,000 tokens; GPT-4 can handle much larger windows.

Training vs. Inference

Training = learning from data, Inference = using the model to answer you.

LLM Architecture and Training Methods (Pre-training, Fine-tuning, RLHF)

Large Language Model Architecture

Component	Description
Tokenisation	Breaks input text into smaller units (tokens) like words or subwords that the model reads
Embedding Layer	Converts tokens into dense vectors (embeddings) capturing semantic meanings
Positional Encoding	Adds information about token order because transformers don't inherently process sequence order

Large Language Model Architecture

Component	Description
Transformer Blocks	Core computational units consisting of:
	Self-Attention: Finds relationships among tokens to understand context and dependencies.
	Multi-Head Attention: Multiple parallel attention mechanisms capture diverse relations.
	Feedforward Neural Network: Processes the attended information for complex pattern recognition.
	Layer Normalisation & Residual Connections: Stabilises training and enables deep stacking.
Stacked Layers	Many transformer blocks stacked to build large models with great expressive power.
Output Layer	Generates probabilities for next token prediction, enabling text generation or classification.

LLM Architecture

Input Layer

- User text is broken into tokens (tiny word pieces)
- Example: “unbelievable” → “un” | “believe” | “able”

Embedding Layer

- Each token becomes a vector (numbers that capture meaning)
- Words with similar meaning sit closer together in this vector space

Transformer Layers (the core engine)

- Dozens/hundreds of layers stacked
- Each layer has:
  - Self-Attention → figures out which tokens are important to each other
  - Feed-forward networks → transform and refine meaning
  - Multiple “attention heads” look at different parts of the text at once

Output Layer

- Predicts the next token step by step
- Strings predictions together into words, sentences and paragraphs

LLM Architecture

Examples

Input Layer

- User types: *“The cat sat on the …”*
- Text is split into tokens → [“The”, “cat”, “sat”, “on”, “the”]

Embedding Layer

- Each token becomes a **vector** (a list of numbers).
- “Cat” sits closer to “dog” in vector space than to “car.”

LLM Architecture

Examples

Transformer Layers

Self-Attention:
- Model looks at how each token relates to the others
- e.g., “sat” relates strongly to “cat”

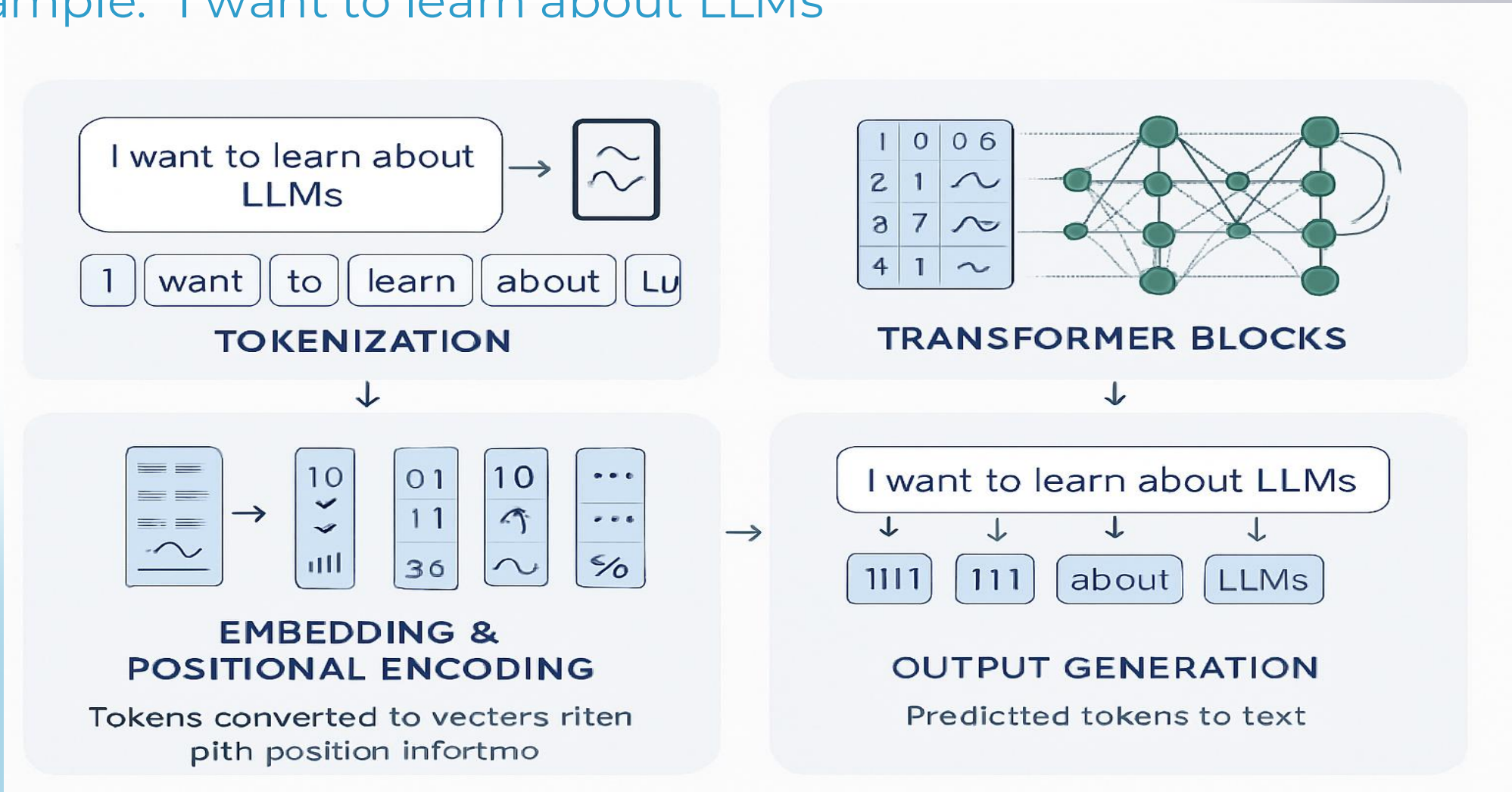
Feed-forward layers:
- Refine signals through stacked layers
- Multiple “attention heads” view relationships from different angles

Output Layer

- Predicts the **most likely next token**
- In this case: “mat” → completes the sentence *“The cat sat on the mat.”*

LLM Architecture Processing

Example: “I want to learn about LLMs”

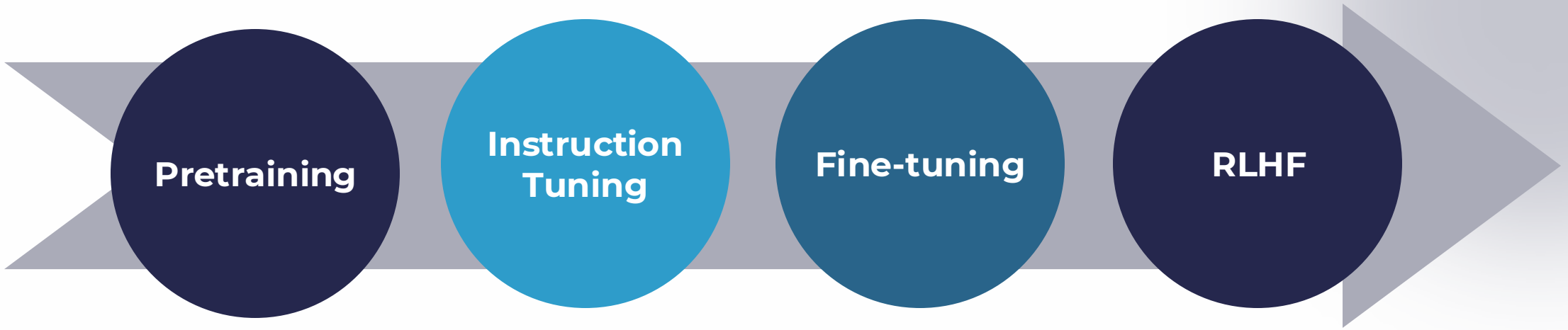


LLM Architecture Processing

Example: “I want to learn about LLMs”

Tokenisation	• Text is split into tokens (small units the model understands). • Example: ["I", " want", " to", " learn", " about", " LL", "Ms"] • Each token maps to a numeric ID.
Embedding + Position	• Tokens are converted into vectors (embeddings) that capture meaning. • Positional encodings are added so the model knows word order.
Transformer Processing	• Embeddings pass through transformer layers. • Self-attention finds relationships between tokens (e.g., “learn” ↔ “LLMs”). • Multi-head attention checks multiple relationships at once.
Decoding / Output Generation	• The model predicts the next token step by step. • Tokens are turned back into words, building the response until finished.

LLMs Training



- Learns from trillions of words (books, web, code)
- Task: predict the next word → builds general language skills

- Trained on tasks with instructions + examples
- Learns to follow directions clearly

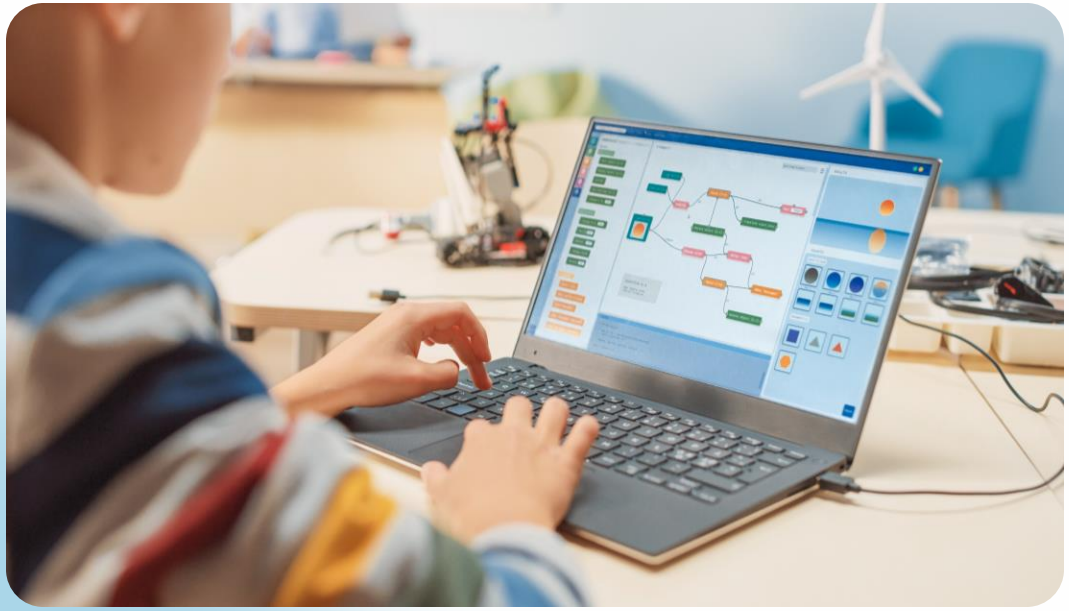
- Adapted for domains (e.g., legal, medical, finance)
- Adds industry-specific expertise

- Humans rank answers → model learns helpful, safe, polite behaviour
- Aligns with human preferences

Pretrain = Learn language | Instruction = Follow tasks | Fine-tune = Specialise | RLHF = Align with humans

Training Methods of (LLMs): Pretraining

Analogy: A child learning language by listening and predicting words without formal teaching.



- Learn from trillions of words across books, articles, web and code
- Predict the next word in a sequence as the core task
- Build a broad sense of grammar, facts and associations

Training Methods of (LLMs):Instruction Tuning

Analogy: A student practicing with a teacher who assigns homework and corrects answers.

- Trains models on datasets with clear instructions + examples
- Teaches the model: “When asked, follow directions.”
- Prompt: “Summarise this paragraph” → Model outputs a short summary

Training Methods of (LLMs):Fine tuning

Analogy: A student choosing a major and specialising after learning the basics.

- Tailors training for specific industries or domains (e.g., medicine, law, finance)
- Adds expertise beyond general knowledge
- **Example:** A general LLM → further trained on medical research papers → becomes a clinical assistant

Training Methods of (LLMs):RLHF

Analogy: A sports player using coaching feedback to adjust and improve performance.

- Humans review model outputs, ranking them as helpful/unhelpful
- Model learns to prefer human-approved answers
- Improves safety, politeness and helpfulness
- **Example:** Instead of bluntly answering “No”, the model replies:
- “I can’t provide that, but here’s something useful instead…”

Inference in LLM Models

Receive new input like a question, prompt or text and generate outputs like answers, summaries or code

Predict next words step by step through the neural network

Apply learned knowledge to real-world tasks — essentially “thinking” in language

Analogy: Inference is like a student using prior learning to answer exam questions.

Limitations, Risks and Future of LLMs

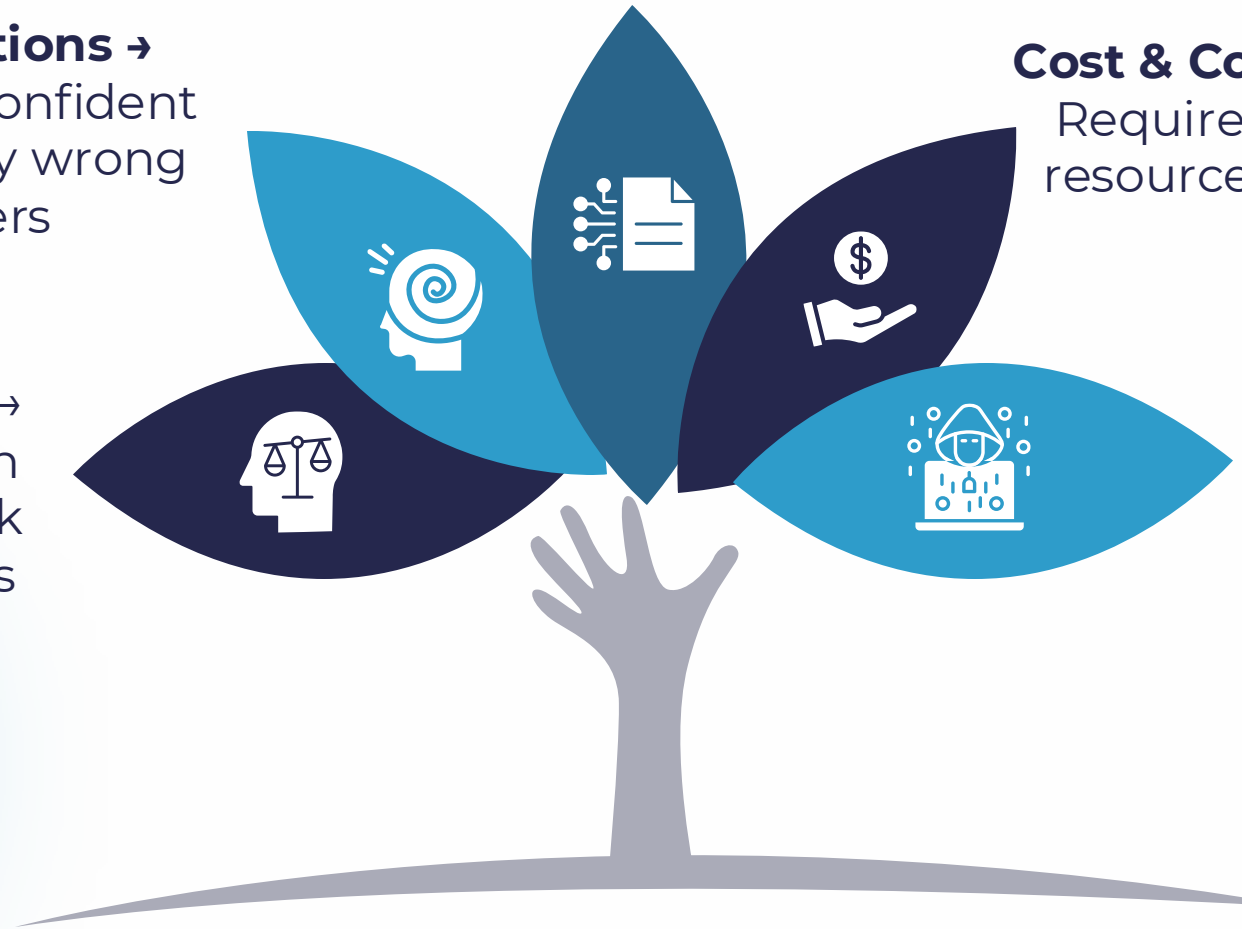
Limitations & Risks of LLMs

Key Challenges

Context Window Limits
→ Only “sees” a fixed chunk of text at once

Hallucinations →
Generates confident but factually wrong answers

Cost & Compute →
Requires heavy resources to run



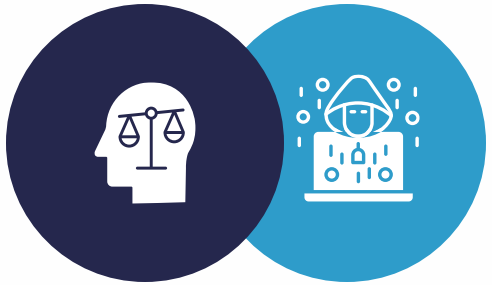
Bias & Fairness →
Reflects biases in training data; risk of unfair outputs

Safety & Misuse →
Potential for harmful, unsafe or unethical use

Hallucinations in LLM

Why it matters

Inherits biases from training data (social, cultural, gender)



Leads to unfair, discriminatory or offensive outputs

Example: Job application assistant ranking resumes unfairly by gender or ethnicity cues.

Hallucinations in LLM

How to handle

- Applies bias audits and fairness checks
- Uses diverse training & evaluation datasets
- Applies Prompt engineering to guide safe outputs
- Uses guardrails on sensitive domains

Bias & Fairness in LLM

Why it matters

Produces confident but incorrect answers

Poses risks for decisions in business, medicine and law

Example: Model invents a fake citation or makes up a “plausible” number.

Bias & Fairness in LLM

How to handle

Use retrieval-augmented generation (RAG) → ground responses in real data

Add verification steps → fact-checking, validators, human review

Never fully automate critical decisions

Context Window Limits in LLM

Why it matters

Sees only a limited number of tokens at once

Forgets earlier context beyond that window

Example: A model loses track of details in a 100-page legal document.

Context Window Limits in LLM

How to handle

Use chunking + smart retrieval (vector databases)

Summarise or compress information before passing it to the model

Use models with larger context windows when available

Cost & Compute in LLM

Why it matters

Running large models is expensive (GPU/TPU costs)

High latency for real-time applications

Example: A customer support bot becomes too costly when scaled to millions of queries.

Cost & Compute in LLM

How to handle

- Use smaller distilled models or quantisation for efficiency
- Cache common queries and responses
- Route requests with lightweight checks before using the LLM

Safety & Misuse in LLM

Why it matters

Generate harmful, misleading or malicious content if misused

Enable phishing, misinformation or unsafe advice

Example: A user prompts the model to generate dangerous instructions.

Safety & Misuse in LLM

How to handle

Apply content filtering and guardrails

Set ethical guidelines and usage policies

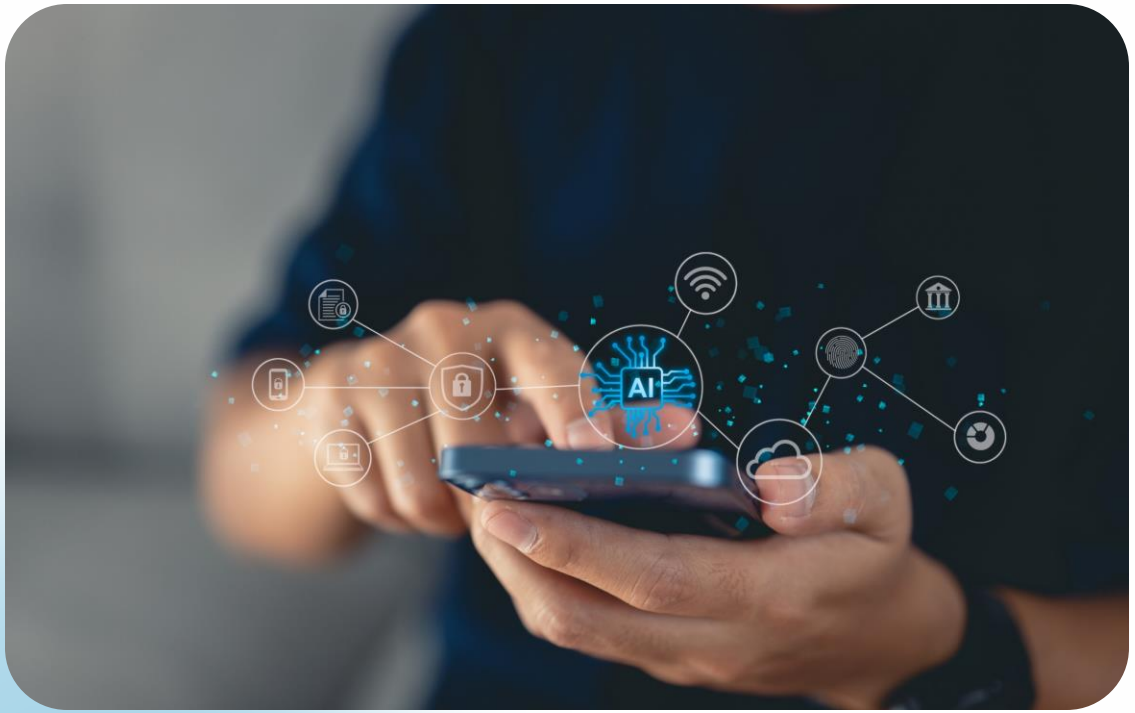
Monitor abuse and add human oversight

History and evolution (from GPT to current models)

Year	Model/Development	Key Features and Milestones
2018	GPT-1	Introduced transformer architecture; pretrained on large text corpora.
2019	GPT-2	1.5 billion parameters; coherent text generation; zero-shot learning.
2020	GPT-3	175 billion parameters; few-shot learning; wide NLP task adaptability.
2021	CLIP, DALL·E	Multimodal models combining vision and language understanding.
2022	Fine-tuning advances	Use of parameter-efficient tuning like LoRA for easier adaptation.
2023	Open-source LLMs	Release of LLaMA, Falcon enabling broader access to powerful LLMs.
2024	GPT-4	Multimodal (text + image); improved alignment and safety features.
2025	RAG, Multimodal LLMs	Integration of retrieval-augmented generation and advanced multimodal AI.

Future of LLMs

LLMs will be smarter, faster, multimodal and key to AGI



- **Multimodal LLMs** (text + images + audio + video) → Richer, natural interactions
- **Smaller, efficient** models (distillation, quantisation, pruning) → Faster, cheaper, on-device use
- **Open vs Proprietary** → Open-source drives innovation; closed models offer polish & safety → Future will be hybrid
- **Role in AGI** → LLMs as stepping stones toward broader intelligence; spark debates on ethics & governance

Q & A

Thank you